

Teaching ‘How Computer/AI Sees’ in Education

Computer vision can be defined as a subfield of artificial intelligence (AI) where machines are trained to interpret and understand visual information. In the context of education, one possible use case is the automation of analysing and grading assignments. However, due to the limitations of computer vision systems, biases and inaccuracies can arise. At the same time, computer vision can also detect details that humans may overlook. For example, handwriting can potentially reveal patterns relating to attentiveness, stress, or emotional state.

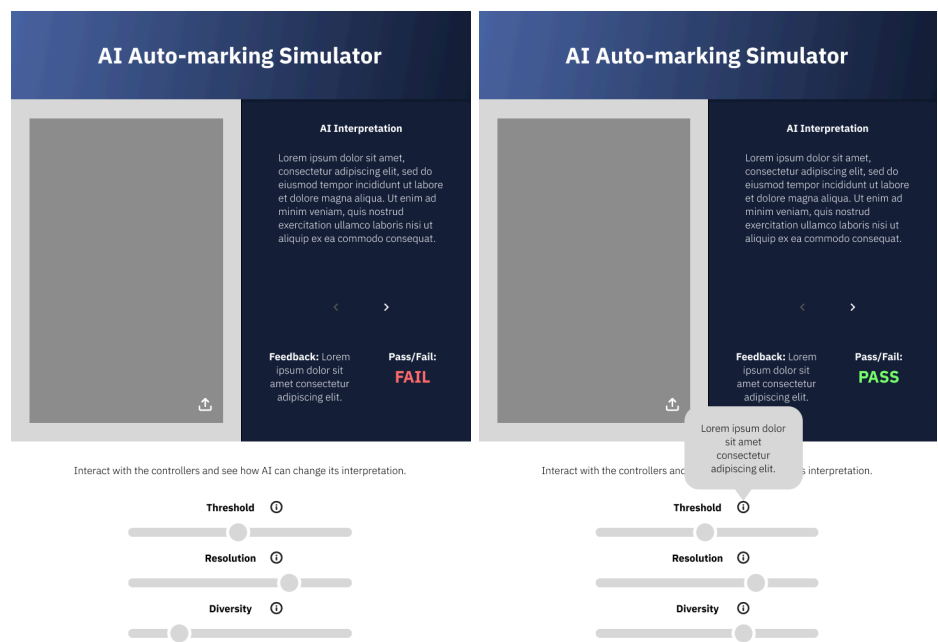
Here, I proposed two simulation mockups that aim to improve AI literacy in an educational context. The target audience is teachers who may use AI tools in their work without having a deep understanding of how these systems function. The first simulation is an auto-marking tool that demonstrates AI bias and how image quality or training diversity can affect AI interpretation. The key takeaway is that AI can speed up processes, but its convenience should not be taken for granted. The second simulation is a note-taking analysis tool that explores how computer vision might be used to approximate student attentiveness and emotional state from handwritten notes. This simulation focuses on showing what computer vision could potentially detect beyond human observation, allowing teachers to draw conclusions about students more quickly through AI-assisted analysis.

When planning these simulations, I first considered the following design question:

“How does the user interaction convey AI literacy subjects without needing lengthy explanations?”

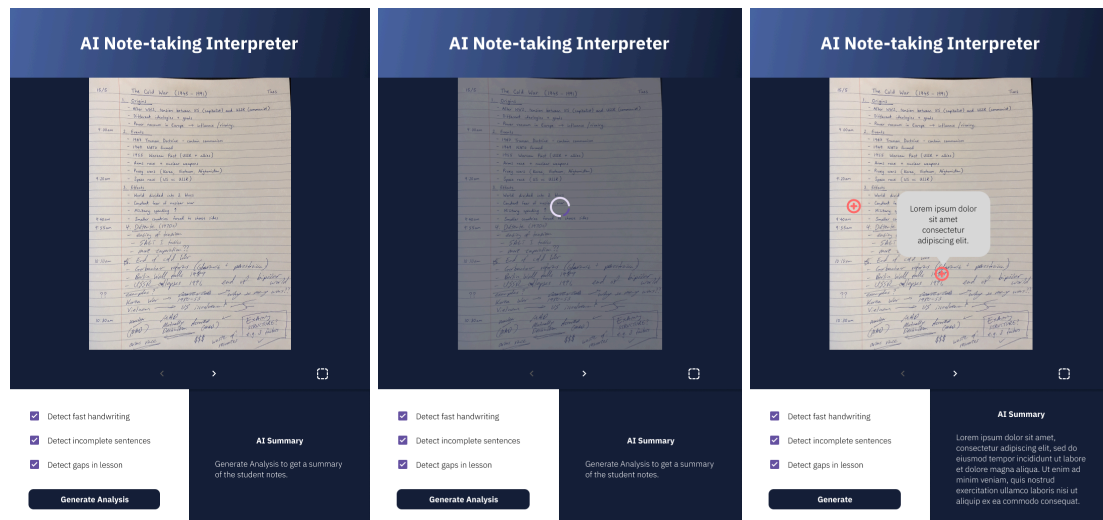
From there, I brainstormed interaction methods that could naturally communicate the ideas behind each simulation. Since Simulation 1 focuses on how image quality and training diversity affect machine performance, I designed the interaction around slider controls that directly manipulate these variables and immediately change the AI output. In contrast, Simulation 2 focuses more on expanding AI capabilities and interpretation, so I decided to use selectable checkboxes that enable or disable different AI analysis features.

After developing the concepts, I designed the user interfaces in Figma before moving into implementation. In Simulation 1, the main interaction revolves around sliders that adjust variables affecting image quality and AI interpretation. To maintain a clean and informative interface, tooltip icons are placed beside each variable label so users can hover over them to better understand the function of each setting.



Simulation 1 UI/UX Design

For Simulation 2, I designed a classroom analysis interface that detects patterns such as attentiveness gaps and emotional indicators from student notes. Users can activate or deactivate different AI analysis features to observe how the system's interpretation changes depending on the enabled capabilities.



Simulation 2 UI/UX Design

Next, I planned the development pipeline for the mockups. Since the simulations did not require fully functional AI models or a real image uploading feature, I used fictional AI-generated student homework and handwritten notes as example assets for the interface. I then designed simple scripted logic to simulate believable AI outputs.

For Simulation 1, the AI feedback changes according to slider thresholds:

- If variables such as image quality, threshold, or dataset diversity are low, the system displays one set of feedback outcomes.
- If the variables are at medium levels, the system displays a different set of outcomes.
- If the variables are high, the AI produces more accurate or confident feedback.

For Simulation 2, the output changes depending on which analysis checkboxes are selected:

- If no checkboxes are selected and the user presses “Generate Analysis”, several default analysis tooltips become visible. These tooltips do not include attributes worth flagging out.
- If any of the specific checkboxes are selected, the corresponding analysis insight becomes active and visible in the interface.

During the development phase, I used LLM tools such as Claude and ChatGPT to assist with parts of the HTML structure and implementation workflow. I would provide the models with exports of my UI designs alongside detailed prompts, then refine the generated code manually based on browser testing and visual adjustments. For example, outside of the AI generated code, I decided to manually implement features such as an image modal/lightbox for expanding the essay PNG. I also used these LLM tools to brainstorm tooltip copy, and simulated AI feedback text.

Here is an example of an initial prompt with Claude:

“Take my design and generate the HTML code for a browser based 2D simulator which educates teachers on AI literacy, tackling the topic of ‘the dangers of over-relying on computer vision AI technology to auto-mark student handwritten assignments’.

Here’s some context:

- The body’s width is maxed out to the rough size of the design uploaded
- Start with the header using the exact word-for-word title and styled with the font, “IBM plex sans”
- `<p>`, `<h1>` (and others) should also use IBM plex sans

- The header has a table below it (2 col, 1 row)
 - The left column contains a placeholder image (upload feature does not need to work yet, however, the upload icon should be placed as a fixed display over the PNG at the bottom right).
 - The right column has a table (1 col, 2 rows). Row 1 is the mock AI interpretation, while row 2 has a table (2 col, 1 row) with mock "Feedback" and "Pass/Fail" results.
- Below the table, add a paragraph with the description "Interact with controllers..."
- After the paragraph are the three interactive sliders with the headers "threshold", "resolution", and "diversity", programmed with Javascript.
- Beside the headers are tooltips. These tooltips are triggered on hover and will make a text bubble with description directly above the icon visible (which is triggered by the Javascript).

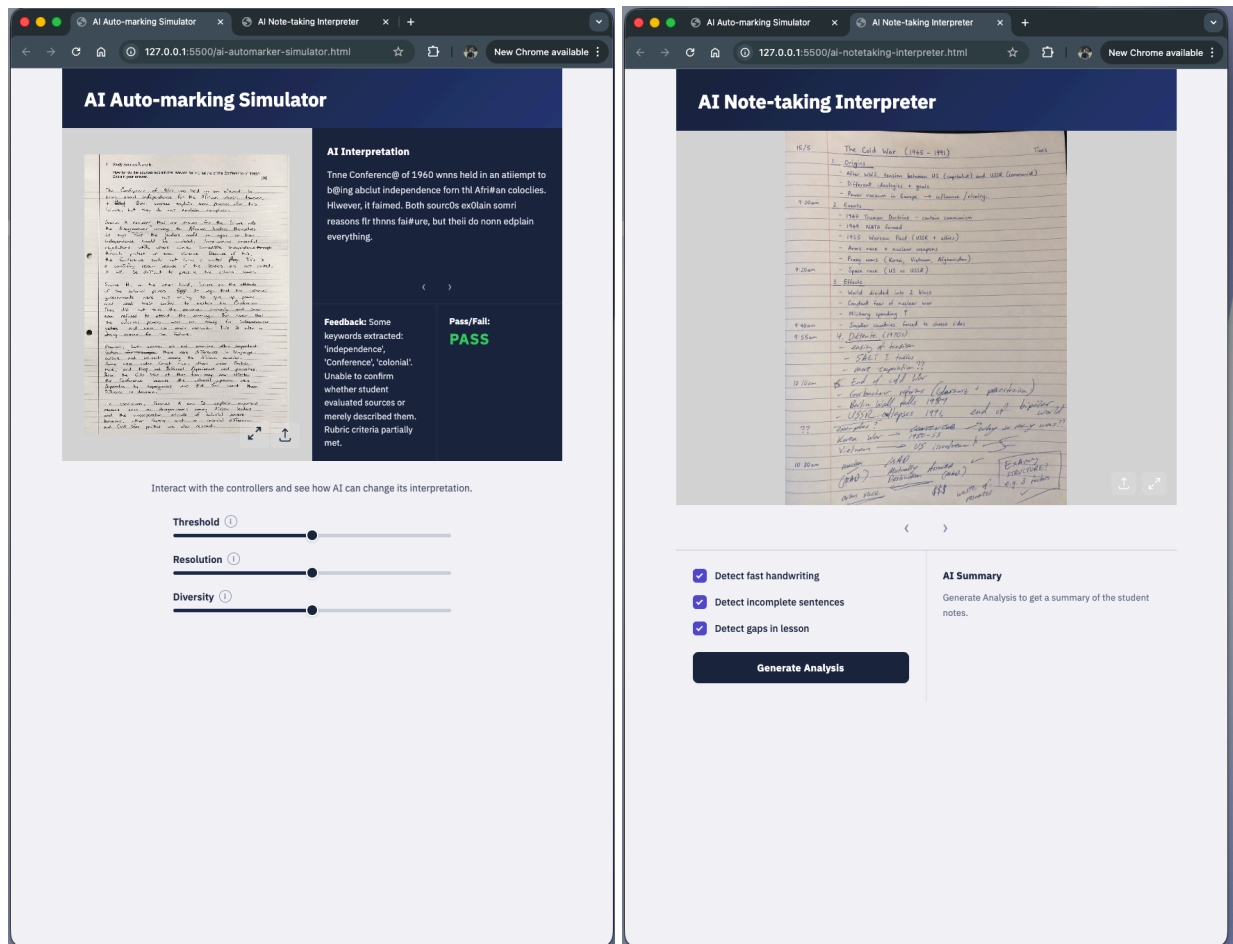
Ideally, when interacting with the sliders, the mock AI Interpretation, Feedback, and Pass/Fail changes.

Since this AI is not real, all the system needs to accomplish is this:

- If low threshold, low resolution, and/or low diversity, Pass/Fail becomes Fail
- If high threshold, high resolution, and/or high diversity, Pass/Fail becomes Pass.

Let's leave it at that for now."

The browser mockups below demonstrate the outcomes of collaborating with the LLM tools during the development process.



Final HTML Mockups

In conclusion, after considering the project timeline and scope, I focused on building high-fidelity simulation mockups that effectively communicate the core educational ideas and interactions behind both concepts. Rather than developing complex AI systems, I relied on simple scripted conditions, UI feedback, and interactive controls to create believable front-end AI behaviour. Through these interactions, the simulations attempt to communicate practical ideas about AI in education in ways that are intuitive and accessible without relying heavily on technical explanations.